D01 · 기획서

promptbench

업무별 정답셋으로 모델 × effort × 프롬프트 조합을 Message Batches API에 한꺼번에 돌려 정확도·비용·속도를 비교하고, 미리 정한 규칙으로 업무별 기본값을 고르는 도구. 과금 호출 전에는 반드시 예상 비용을 보여 주고 승인을 받는다.

문서 버전	1.0
작성일	2026-10-20
작성자	김태민
문서 상태	구현·실측 완료본

변경 이력

버전	일자	내용	작성자
1.0	2026-10-20	최초 작성	김태민

목차

1. 배경과 문제 정의
2. 현재 대안 비교
3. 대상 사용자
4. 핵심 가치
5. 사용 흐름
6. MVP 범위와 제외
7. 성공 지표
8. 리스크와 대응
9. 데이터 한계
10. 로드맵

표기 원칙 — 목표치 실측 전에 정한 목표(실측 값은 실측 리포트 D09에 있다) · 확인함 구현 1단계에서 실제로 확인한 Anthropic API 동작. 이 표시가 없는 사실 진술은 출처가 있거나 직접 확인한 것이다. 문서 간 참조는 공통 ID(`CMD-nn` 명령 · `SN-nn` 화면 · `POL-nn` 정책 · `ADR-nn` 결정 · `ST-P-nn` 계획 상태 · 업무 `T1` ~ `T3`)로 하며, 끊긴 참조는 빌드 단계에서 자동 검사한다.

1. 배경과 문제 정의

1.1 한 줄 문제

> 업무에 Claude를 붙일 때 "어느 모델에, effort는 얼마로, 프롬프트는 어떤 버전으로"를 정하는 근거가 없다.

이 기본값은 흔히 감이나 한두 건 시험으로 정해진다. 문제는 세 가지다.

- **비싼 모델이 늘 필요한 것은 아니다.** 업무에 따라 싼 모델 + 높은 effort나 프롬프트 개선만으로 충분할 수 있다. 하지만 이를 **숫자로 보여 주는 절차**가 없다.
- **조합 수가 많다.** 모델 3 × effort 5 × 프롬프트 2 × 업무 3 = 90 조합. 전부 손으로 돌릴 수 없다.
- **평가 자체에도 돈이 든다.** 평가 비용을 미리 알고 통제하지 못하면 평가를 하지 않게 되고, 다시 감으로 돌아간다.

1.2 기획 질문

업무별 정답셋으로 조합을 한꺼번에 비교하고, 평가 비용은 미리 알고 통제하면서, 기본값을 근거와 함께 고를 수 있는가?

> promptbench는 업무별 정답셋으로 모델 × effort × 프롬프트 조합을 Message Batches API에 한꺼번에 돌려 정확도·비용·속도를 비교하고, 미리 정한 규칙으로 업무별 기본값을 고른다. 과금 호출 전에는 반드시 예상 비용을 보여 주고 승인을 받는다.

1.3 포지셔닝 — 범위를 좁힌다

선택지	판단
범용 평가 프레임워크	✗ 범위가 넓다. 여러 공급자 지원은 이 도구의 목적이 아니다
Claude 모델끼리 비교하는 업무 기본값 결정 도구	✅ 범위를 좁힌다. 모델 호출부는 어댑터로 분리해 확장 여지만 남긴다 (ADR-02)
Console Workbench 등 수작업 비교	소수 시험엔 좋지만 정답셋 전체 채점·비용 통제·재현이 어렵다고 본다

핵심 메시지: "좋은 모델을 쓰자"가 아니라 "이 업무엔 이 조합이면 충분하다"를 비용과 함께 증명한다. 예산 제약 자체를 기능(표본 배분 + 승인 게이트)으로 바꾼다.

1.4 결제 전제

- 실측은 **Console API 크레딧**(`ANTHROPIC_API_KEY`)으로 한다. Claude 구독과는 별개다(POL-16).
- **실측 전체 예산 상한은 $5**다. 파일럿 + 본실행 + 채점 + 속도 측정의 합계이며, 실행 한 번이 아니라 프로젝트 누적 장부에 걸린다(POL-01).
- 이름 `promptbench` 는 PyPI에서 Microsoft의 무관한 프로젝트가 점유하고 있다(2026-10-20 확인). 그래서 MVP는 PyPI에 배포하지 않고 로컬 설치만 한다.

2. 현재 대안 비교

세 대안 모두 일반적인 사용 방식에 대한 정리이며 특정 제품의 기능 목록이 아니다. 개별 도구의 기능은 직접 검증하지 않았다. 시장 규모·사용자 수 같은 수치는 싣지 않는다. 범용 평가 프레임워크의 개별 기능은 제품마다 달라 이 문서에서 단정하지 않고, 아래 4개 기준에서 promptbench가 노리는 칸만 적는다.

기준	감 · 한두 건 시험	Console Workbench 수작업	범용 평가 프레임워크	promptbench
정답셋 전체 채점	없음 — 눈으로 몇 건 확인	소수 건을 눈으로 비교. 전체 채점은 직접 해야 함	정답셋 채점은 가능한 쪽	업무별 채점기로 전체 표본을 자동 채점 (POL-11)
비용 사전 통제	없음	호출할 때마다 과금. 전체 금액을 미리 보는 절차 없음	도구마다 다름	예상·보수·최악 금액을 승인 화면에 보여 주고 상한을 넘기면 제출 전 정지 (POL-03, POL-07)
재현성	없음 — 무엇을 돌렸는지 남지 않음	화면 상태에 의존	설정 파일 기반으로 재현 가능한 쪽	고정 시드 표본 ID · 계획 해시 · 원본 응답 보관 (POL-04, ADR-05)
결정 근거 문서화	없음	수작업으로 메모	결과표까지, 선정 기준은 사용자 몫	선정 규칙을 실행 전에 고정하고 결과를 리포트로 남김 (POL-13, POL-14)

정리: 수작업은 비용 통제와 재현이 약하고, 범용 프레임워크는 범위가 넓어 "이 업무의 기본값은 이것"이라는 결정까지 이어 주지 않는다. promptbench는 **결정 하나를 근거와 함께 내는 데까지** 범위를 좁힌다.

3. 대상 사용자 (페르소나)

아래 두 유형은 도구의 설계 기준으로 삼은 상정 사용자이며, 실제 사용자 조사 결과가 아니다.

P1. 사내에 LLM 업무 자동화를 도입하는 기획자

항목	내용
상황	업무에 Claude를 붙이려 하고, 모델·설정을 근거와 함께 정해 보고해야 한다
지금 하는 일	몇 건 시험해 본 인상으로 모델을 고르거나, 가장 좋은 모델을 기본으로 쓴다
원하는 것	"이 업무엔 이 조합이면 충분하다"는 숫자와 비용 근거
promptbench에서의 가치	선정 규칙과 결과표를 그대로 보고 자료로 쓴다(`promptbench report` , CMD-07, SN-09)

P2. API 비용을 통제하며 프롬프트를 개선하는 소규모 팀 개발자

항목	내용
상황	프롬프트를 고칠 때마다 좋아졌는지 확신이 없고, 평가용 호출 비용도 부담이다
지금 하는 일	소수 입력으로 눈으로 비교하거나, 평가 없이 배포한다
원하는 것	돈이 얼마 드는지 먼저 보고, 승인한 만큼만 쓰기

항목	내용
promptbench에서의 가치	승인 화면에서 예상 금액을 확인하고(SN-03) 장부로 누적 사용액을 본다(`promptbench status` , CMD-05, SN-10)

4. 핵심 가치

#	핵심 가치	구현 방식	근거
V1	**근거 있는 기본값**	업무별 정답셋으로 조합을 채점하고, 실행 전에 고정한 규칙(품질이 동급인 후보 중 1,000건당 비용이 가장 낮은 조합)으로 기본값을 고른다	POL-11, POL-13, POL-14
V2	**평가 비용 통제**	예상 비용을 먼저 보여 주고 승인받는다. 승인 전 과금 호출은 0건, 다음 배치가 상한을 넘기면 제출 전에 멈춘다	POL-01, POL-03, POL-07
V3	**재현 가능한 기록**	표본은 고정 시드, 계획은 해시로 고정, 실제 비용은 응답 `usage` 로 다시 계산해 장부에 쌓고 예상치와 비교한다. 원본 응답을 보관해 다시 채점할 수 있다	POL-02, POL-06, ADR-05

원칙 — 결과를 본 뒤 기준을 바꾸지 않는다. 선정 규칙과 허용폭은 실행 전에 적어 둔다. 그래야 "이긴 조합"이 아니라 "규칙이 고른 조합"이라고 말할 수 있다.

5. 사용 흐름

#	단계	명령 / 화면	담당	결과
1	준비	`promptbench prepare` (CMD-01 · SN-01)	코드	데이터를 내려받고 고정 시드로 표본을 나눠 표본 ID 목록을 기록
2	계획	`promptbench plan --stage …` (CMD-02 · SN-02, SN-04)	코드	조합 × 표본 배분. 예상·보수·최악 금액을 계산하고, 예산에 맞춰 표본을 줄인다. 줄여도 안 되면 이유를 내고 계획을 만들지 않는다
3	승인	`promptbench approve` (CMD-03 · SN-03)	사람	조합표·건수·금액 3종·누적 사용액을 보고 `y` . 승인은 계획 해시에 묶이며 계획이 바뀌면 다시 받는다 (ST-P-01 → ST-P-02)
4	실행	`promptbench run` (CMD-04 · SN-05, SN-06, SN-07)	코드	작은 배치로 나눠 순서대로 제출·수집. 상한이 임박하면 제출 전에 정지 (ST-P-05), `--resume` 으로 이어간다
5	보고	`promptbench report` (CMD-07 · SN-09)	코드	조합별 결과표 · 파레토 그림 · 기본값 선정 → D09 원본

- **계획 상태**: 초안 → 승인 → 실행 중 → 완료, 또는 정지(ST-P-01 ~ ST-P-05). 정지된 계획은 같은 해시로 `run --resume` 해 이어 가며(남은 예산 ≥ 다음 배치 보수 금액일 때, 새 승인 없음 — POL-08), 실패 항목 재시도만 새 계획으로 다시 승인받는다(POL-10).
- **실험 단계**: pilot(출력 토큰 측정) → R1(Haiku, 프롬프트 v1 · v2) → judge-R1(R1의 T3 공통 30건 채점 — T3의 이긴 프롬프트를 정하는 데 필요) → R2(Sonnet · Opus, R1의 Haiku 조합에서 이긴 프롬프트 1개) → judge-R2(R2의 T3 공통 30건 채점) → latency(속도 측정) → calibrate(사람 교정, 과금 없음, CMD-06 · SN-08 — judge-R2 뒤면 언제든) → report. 과금 단계마다 같은 승인 게이트를 거친다.

- **배치의 성격**: Message Batches는 50% 할인이며, 대부분 1시간 안에 끝나지만 최대 24시간 걸릴 수 있고 24시간 안에 끝나지 않으면 만료된다. 결과는 요청 순서와 다르게 올 수 있어 `custom_id`로 매칭한다(공식 문서, 2026-10-20 확인).
- **조합 수**: 업무 3개(T1 문의 분류 · T2 정보 추출 · T3 요약)를 모델 3종 × effort로 돌린다. 전체 90 조합을 다 돌리지 않고, 표본 배분(Haiku effort 5단계 × main 100건 × v1 · v2 · Sonnet low/medium × 공통 30건 · Opus low × 공통 30건, Sonnet · Opus 는 이긴 프롬프트 1개)으로 요청 수를 줄인다. T3 채점은 공통 30건만 Sonnet 5.5 · high 로 한다. 이 배분은 $5 상한 안에 들도록 2026-10-20 사용자가 정했다 — 계산 합계와 남는 예비는 D03 2장 · D05 1-3 · D08 7-1 의 계산표에 있다.

6. MVP 범위와 제외

6.1 포함

영역	포함 기능	근거
업무	T1 문의 분류 · T2 정보 추출 · T3 요약, 업무별 프롬프트 v1·v2와 채점기	POL-11, POL-12
비교 대상	Claude 3종(`claude-haiku-5-5` · `claude-sonnet-5-5` · `claude-opus-5-5`) × effort(`low` ~ `max`) × 프롬프트 버전	ADR-02, ADR-08
실행	Message Batches 분할 제출 · 결과 수집 · 재개 가능한 실행 · 속도는 소표본 동기 스트리밍	ADR-03, ADR-04
비용 통제	`count_tokens` 기반 예상 · 승인 게이트 · 누적 장부 · 상한 정지 · 가격표 확인일 경고	POL-01~POL-08, POL-18, POL-19
채점·선정	업무별 지표 · T3 LLM 채점자 + 사람 교정 · 사전 등록된 선정 규칙	POL-11~POL-14
기록	SQLite 장부 + 원본 JSONL · 리포트(결과표 · 파레토 그림 · 예상 대 실제 비용)	ADR-05, CMD-07
인증·검증	`ANTHROPIC_API_KEY` 하나만 쓰고 키를 어디에도 남기지 않음 · 기본 테스트는 FakeAdapter로 API 호출 0건	POL-16, POL-17, POL-03
입구	CLI 하나, 명령 7개(`prepare` · `plan` · `approve` · `run` · `status` · `calibrate` · `report`)	CMD-01~CMD-07

6.2 제외 (향후 계획)

제외 항목	제외 이유	다시 검토할 조건
다른 공급자	이 도구의 목적은 Claude 모델끼리의 비교다. 호출부는 어댑터 뒤에 두어 확장 여지만 남긴다 (ADR-02)	다른 공급자 비교 요구가 반복될 때
웹 UI	결정 하나를 내는 도구라 CLI로 충분하다	CLI 리포트로는 결과를 공유하기 어렵다는 요구가 반복될 때
프롬프트 자동 개선	이 도구는 주어진 버전을 비교할 뿐 새 프롬프트를 만들지 않는다	비교 결과로 개선 방향이 일관되게 보일 때
PyPI 배포	`promptbench` 이름이 이미 다른 프로젝트에 점유됨. MVP는 로컬 설치만 한다	공개 배포 요구가 생기고 패키지명이 정해질 때

7. 성공 지표

목표치 목표 열은 실측 전에 정한 **목표**다. 실측 결과는 **D09 실측 리포트**에 있고, 아래 표의 "실측(D09)" 열이 달성 여부다. 목표를 못 맞춘 경우도 정직하게 싣는다.

#	지표	정의	목표 목표치	측정 방법 (D09)	실측(D09)
M1	실측 총비용	파일럿 + 본실행 + 채점 + 속도 측정의 실제 비용 합계 (응답 `usage` 기준 장부)	**$5 이하**	누적 장부의 최종 합계 (POL-01, POL-02)	장부 **$4.3709** — 달성. 콘솔 이번 달 지출은 latency 전 $5.12(장부 $4.1196)로 약 $1.00 차이가 있고 원인은 확인하지 못했다
M2	예상 대 실제 비용 오차	(실제 − 예상) / 예상. 계획기의 예상치(파일럿 평균 기준)와 실제 청구 비용 비교	**±30% 이내**	단계별로 예상과 실제를 나란히 리포트 (POL-06)	파일럿 이후 5단계 −3.7% ~ +10.2% — 달성. 파일럿은 설계 단계 가정으로 예상해 −92.5%
M3	기본값 확정	업무 3개(T1~T3) 각각이 사전 등록된 선정 규칙으로 기본값을 낸 경우	**3개 모두**	리포트의 선정 결과. 공통 30건 기준이므로 확정에는 추가 표본이 필요하다는 단서를 함께 싣는다 (POL-13, POL-14)	업무 3개 모두 선정 — 달성. T3 은 채점자 교정 일치율 63.3% 로 참고용, 모두 공통 30건 기준
M4	승인 없는 과금 호출	승인 전, 또는 승인한 계획과 다른 계획으로 나간 과금 호출	**0건**	게이트 테스트(어댑터 `submit` 호출 횟수) + 장부와 승인 기록 대조 (POL-03, POL-04)	과금 계획 6개 모두 사용자 승인 뒤 실행 — 계획 기준 0건

M1·M4를 성과로 둔 이유: 정확도 비교만 잘해도 평가 비용이 통제되지 않으면 평가를 못 돌린다. 이 도구의 가치는 "싸고 안전하게 근거를 만든다"까지다.

8. 리스크와 대응

#	리스크	영향	대응	관련
R1	**30건 표본의 넓은 신뢰구간** — 공통 표본이 30건이라 조합 간 차이가 구간에 묻힌다	"동급" 판정이 넓게 나와 기본값이 부정확해질 수 있다	짝지은 부트스트랩으로 구간을 보이고, 허용폭을 실행 전에 고정한다. 리포트에 "공통 30건 기준 판정 — 확정에는 추가 표본 필요"를 붙인다. Haiku 내부 비교는 100건으로 더 정밀하게 한다	POL-13, POL-14
R2	**LLM 채점자 편향** — T3 채점자가 Sonnet 5.5이므로 Sonnet의 요약을 후하게 줄 수 있고, 가장 강한 모델(Opus)보다 판정이 약할 수 있다	T3 결과의 신뢰도 하락	어느 모델의 답인지 숨긴다(블라인드 — 요약을 내는 순서를 섞는 것은 운영상의 선택일 뿐 편향 대책이 아니다. Batch 요청은 서로 독립이다). 사람이 30개를 교정하고 일치율과 Cohen's κ를 보고한다. 통과/실패 일치율이 80% 미만이면 T3를 "참고용"으로 표시하고 한계로 남긴다	POL-12, POL-22

#	리스크	영향	대응	관련
R3	**데이터셋과 실제 업무의 차이** — 공개 데이터가 사용자의 업무와 다르다	선정된 기본값이 실제 업무에서 달라질 수 있다	9장에 한계를 명시한다. 이 도구의 가치는 특정 결과 수치가 아니라 **절차**(같은 방법으로 자기 정답셋을 넣어 다시 돌릴 수 있음)임을 분명히 한다	POL-15
R4	**API 동작 미확인** — 구현해 보기 전에 확인하지 못한 동작이 있었다	계획기의 가정이 틀리면 비용 예상이 어긋난다	아래 항목을 구현 1단계에서 실측해 확인했다. 틀린 가정이 있었다면 설계 문서에 반영하는 것이 원칙이었다	POL-05, POL-06
R5	**배분이 예산 상한에 빠듯함** — 처음 배분은 설계 단계 가정으로 계산하면 $5 를 크게 넘어(특히 T3 채점), 2026-10-20 사용자 결정으로 채점 범위 · 채점 모델 · R2 조합 · 속도 측정 건수를 줄였다. 계산 합계는 상한 안이지만 실제 토큰이 가정보다 크면 뒤 단계에서 모자랄 수 있다	judge-R2 · latency 단계에서 예산이 바닥난다	상한에서 계산 합계를 뺀 나머지를 예비로 남긴다(D03 2장). 단계마다 보수 금액 ≤ 남은 예산일 때만 계획을 만들고, R1 은 Haiku 비공통 표본, latency 는 3 → 2 → 1건 순으로만 줄이며 채점 · R2 는 줄이지 않는다(POL-21). 파일럿 뒤 실제 출력 통계로 다시 계산한다	POL-01, POL-21

R4에서 확인한 항목 확인함 (구현 1단계 · 2026-10-21, 결과는 `docs/superpowers/notes/2026-10-21-api-checks.md`)

- Batch 요청에서 `output_config.effort` 와 `output_config.format` (structured outputs)을 함께 쓸 수 있는가 — **가능.** Haiku 5.5 Batch 2건이 둘 다 `succeeded` 이고 응답이 스키마를 지켰다.
- `count_tokens` 가 출력 스키마 · effort 설정을 받는가, 스키마 토큰이 입력에 포함되는가 — **받는다.** 스키마 토큰(+213)이 입력에 포함되고 effort 는 입력 토큰에 영향이 없다.
- Haiku 5.5의 10만 토큰 초과 가격 구간 — 확인 대상에서 빼고, 계획마다 `long_context_requests` 경고로 검사하도록 구현했다.
- Batch 결과의 `usage` 에 thinking 토큰이 출력 토큰으로 포함되는가 — **포함된다**(근사 확인). 콘솔 청구와 장부 합계에는 약 $1 의 차이가 있고 원인은 확인하지 못했다(D09 4장).

9. 데이터 한계

데이터 원본은 저장소에 넣지 않는다. 내려받기 스크립트와 **고정 시드로 뽑은 표본 ID 목록**만 커밋한다. 라이선스는 2026-10-20에 각 데이터셋 카드에서 확인했다(POL-15).

업무	데이터셋	라이선스	규모	채점
T1 문의 분류	MASSIVE ko-KR 의도 60종	CC BY 4.0	ko-KR train 11,514 · dev 2,033 · test 2,974건	정확도 · macro-F1
T2 정보 추출	MASSIVE ko-KR 슬롯 주석 55종 → JSON 필드	CC BY 4.0	T1과 같은 발화	필드 단위 P/R/F1 · 레코드 완전 일치율
T3 요약	XL-Sum Korean (BBC 코리아 기사)	CC BY-NC-SA 4.0 (비상업 연구용)	Korean train 4,407 · dev 550 · test 550건	LLM 채점자 루브릭 + 사람 교정

출처: `huggingface.co/datasets/AmazonScience/massive` , `huggingface.co/datasets/csebuetnlp/xlsum` .

한계

- **MASSIVE는 음성 비서 발화다.** 실제 고객 문의와 문체·길이가 다르다. T1·T2 결과를 곧바로 "고객 문의 분류의 최적 설정"으로 읽으면 안 된다 (R3).
- **XL-Sum은 비상업 라이선스다.** 포트폴리오·연구용으로만 쓰고 원문을 재배포하지 않는다. 저장소에는 표본 ID 목록만 둔다.
- **T1·T2는 같은 발화를 쓴다.** "같은 입력, 다른 업무"에서 기본값이 어떻게 달라지는지 비교할 수 있지만, 두 업무의 결과가 독립적이지는 않다.
- **표본이 작다.** 업무당 main 100건, 그중 앞 30건이 모든 조합의 공통 표본이다(R1). pilot 3건은 출력 토큰 측정에만 쓰고 품질 집계에는 쓰지 않는다.
- **T3 채점은 LLM에 기댄다.** 사람 교정 30개로 일치도를 확인하지만 표본이 작고, 채점자 Sonnet 5.5 의 Sonnet 요약에 대한 자기 선호 편향 가능성이 남는다(R2). T3 품질은 공통 30건에서만 재므로 Haiku T3 의 effort 곡선도 30건 기준이다.

10. 로드맵

단계	내용	산출물	상태
계획 A — 기획	문제 정의, 요구사항, 정책, 규격, IA, CLI 화면 설계, 기능명세, 시스템·AI 설계, 빌드 도구	D01~D08 + README	완료
계획 B — 구현	업무 정의 · 계획기 · 추정기 · 게이트 · 어댑터 · 실행기 · 채점기 · 저장소 · 리포트 · CLI · 테스트	동작하는 `promptbench` (로컬 설치, 배포 안 함)	완료
계획 C — 실측과 마무리	pilot → R1 → judge-R1 → R2 → judge-R2 → latency → calibrate → report 실측, 요약본	D09 실측 리포트, D10 요약본, README 갱신	완료

★ 계획 C의 실측은 모두 승인 게이트를 거쳤고, 장부 누적 $4.3709 로 상한 $5 를 넘기지 않았다. 배분은 2026-10-20 사용자 결정으로 확정했다(D03 2장).